

LAPORAN PRAKTIKUM SISTEM OPERASI

Dosen Pengampu :

TRIAWAN ADI CAHYANTO M.Kom



Disusun Oleh:

Pradana Bagus Susilo (2410651007)

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

2025

Tujuan

Process Concept dan Process Operations:

1. Memahami konsep proses dan siklus hidupnya
2. Mampu membuat dan mengelola proses menggunakan system call
3. Memahami hubungan parent-child process
4. Menganalisis Process ID (PID) dan Process Control Block

Dasar Teori

1. Konsep Proses (Process Concept)

Proses merupakan program yang sedang dieksekusi oleh sistem operasi. Setiap proses memiliki status eksekusi, ruang memori, dan sumber daya yang diperlukan untuk menjalankan instruksi. Sistem operasi bertanggung jawab untuk membuat, menjadwalkan, mengatur, dan mengakhiri proses agar pemanfaatan CPU dan memori dapat dilakukan secara efisien.

Sebuah proses memiliki beberapa komponen utama, antara lain:

- Program Counter: Menyimpan alamat instruksi berikutnya yang akan dieksekusi.
- Stack: Menyimpan data sementara seperti parameter fungsi, alamat pengembalian, dan variabel lokal.
- Data Section: Menyimpan variabel global.
- Heap: Menyimpan data dinamis yang dialokasikan selama proses berjalan.

Selain itu, setiap proses dikelola oleh Process Control Block (PCB), yaitu struktur data yang menyimpan informasi penting seperti ID proses, status, register CPU, serta informasi manajemen memori.

2. IPC Methods (Inter-Process Communication Methods)

Inter-Process Communication (IPC) adalah mekanisme yang digunakan untuk berkomunikasi dan bertukar data antar proses. IPC penting terutama dalam sistem multiprogramming atau multiprocessor, di mana beberapa proses perlu bekerja sama untuk menyelesaikan suatu tugas.

Ada dua kategori utama dalam metode IPC:

A. Direct Communication

Proses saling berkomunikasi secara langsung dengan menyebutkan nama proses tujuan.

- Setiap pesan dikirim dari proses pengirim ke proses penerima tertentu.
- Biasanya digunakan pada sistem dengan jumlah proses yang terbatas.

Contoh: `send(P1, message)` dan `receive(P2, message)`.

B. Indirect Communication

Komunikasi dilakukan melalui media perantara yang disebut mailbox atau message queue.

- Pesan dikirim ke mailbox, dan proses penerima mengambil pesan dari sana.
- Lebih fleksibel karena tidak perlu tahu identitas proses lain

Topik 1

1.1 Membuat Proses Sederhana

Source Code:

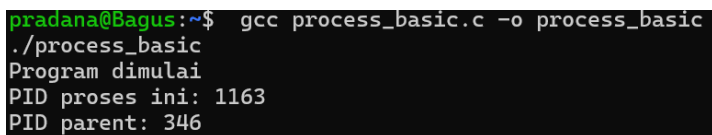
```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main() {
    pid_t pid;

    printf("Program dimulai\n");
    printf("PID proses ini: %d\n", getpid());
    printf("PID parent: %d\n", getppid());

    return 0;
}
```

Output:

A terminal window with a black background and green text. The prompt is 'pradana@Bagus:~\$'. The command 'gcc process_basic.c -o process_basic' is entered. The next line shows the execution './process_basic'. The output consists of three lines: 'Program dimulai', 'PID proses ini: 1163', and 'PID parent: 346'.

```
pradana@Bagus:~$ gcc process_basic.c -o process_basic
./process_basic
Program dimulai
PID proses ini: 1163
PID parent: 346
```

Analisis Hasil:

- Program kamu (PID 1234) sedang berjalan sebagai proses aktif.
- Proses parent-nya (PID 1200) adalah bash shell atau terminal yang menjalankannya.
- Setiap kali dijalankan ulang, PID-nya bakal beda, karena sistem bikin proses baru setiap eksekusi.

1.2 Fork - Parent dan Child Process

Source Code:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main() {
    pid_t pid;

    printf("Sebelum fork()\n");
    printf("PID: %d\n\n", getpid());

    pid = fork();
    if (pid < 0) {
```

```

        fprintf(stderr, "Fork gagal\n");
        return 1;
    }
    else if (pid == 0) {
        // Child process
        printf("CHILD PROCESS\n");
        printf("PID saya: %d\n", getpid());
        printf("PID parent saya: %d\n", getppid());
    }
    else {
        // Parent process
        printf("PARENT PROCESS\n");
        printf("PID saya: %d\n", getpid());
        printf("PID child saya: %d\n", pid);
    }

    printf("Proses %d selesai\n\n", getpid());
    return 0;
}

```

Output:

```

pradana@Bagus:~$ gcc fork_demo.c -o fork_demo
./fork_demo
Sebelum fork()
PID: 1205

PARENT PROCESS
PID saya: 1205
PID child saya: 1206
Proses 1205 selesai

CHILD PROCESS
PID saya: 1206
PID parent saya: 1205
Proses 1206 selesai

```

Analisis Hasil:

- fork() membuat dua proses dengan eksekusi terpisah.
- Child dan Parent punya salinan memori yang sama saat awal, tapi independen setelah fork.
- Kedua proses bisa berjalan bersamaan (tergantung scheduler OS).
- Menunjukkan konsep proses paralel (multitasking) di sistem operasi.

1.3 Process Termination dengan wait()

Source Kode:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid;
    int status;

    pid = fork();

    if (pid == 0) {
        // Child process
        printf("Child: PID saya %d, akan tidur 3 detik\n", getpid());
        sleep(3);
        printf("Child: Selesai!\n");
        return 42; // Exit code
    } else {
        // Parent process
        printf("Parent: Menunggu child (PID: %d) selesai...\n", pid);
        wait(&status);

        if (WIFEXITED(status)) {
            printf("Parent: Child selesai dengan exit code: %d\n", WEXITSTATUS(status));
        }
    }

    return 0;
}
```

Output:

```
pradana@Bagus:~$ gcc process_wait.c -o process_wait
pradana@Bagus:~$ cd./process_wait
-bash: cd./process_wait: No such file or directory
pradana@Bagus:~$ ./process_wait
Parent: Menunggu child (PID: 1301) selesai...
Child: PID saya 1301, akan tidur 3 detik
Child: Selesai!
Parent: Child selesai dengan exit code: 42
pradana@Bagus:~$ |
```

Analisis Hasil:

- Parent tidak langsung lanjut eksekusi, tapi menunggu child selesai berkat wait().
- Tanpa wait(), parent bisa selesai lebih dulu, dan child jadi “proses yatim (orphan)” yang nanti diadopsi oleh init/systemd.
- WIFEXITED(status) memastikan child selesai secara normal (bukan error).
- WEXITSTATUS(status) ambil nilai yang dikembalikan oleh return di child (42 dalam hal ini).

1.4 Multiple Child Processes

Source Kode:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int i;
    pid_t pid;
    int num_children = 3;

    printf("Parent PID: %d\n\n", getpid());

    for (i = 0; i < num_children; i++) {
        pid = fork();

        if (pid == 0) {
            // Child process
            printf("Child %d: PID = %d, Parent PID = %d\n", i + 1, getpid(),
getppid());
            sleep(i + 1); // Simulasi pekerjaan
            printf("Child %d selesai!\n", i + 1);
            return 1;
        }
    }
}
```

```

// Parent menunggu semua child
for (i = 0; i < num_children; i++) {
    int status;
    pid_t child_pid = wait(&status);
    printf("Parent: Child dengan PID %d selesai\n", child_pid);
}

printf("\nParent: Semua child selesai\n");

return 0;
}

```

Output:

```

pradana@Bagus:~$ gcc multiple_fork.c -o multiple_fork
pradana@Bagus:~$ ./multiple_forl
-bash: ./multiple_forl: No such file or directory
pradana@Bagus:~$ ./multiple_fork
Parent PID: 1354

Child 1: PID = 1355, Parent PID = 1354
Child 2: PID = 1356, Parent PID = 1354
Child 3: PID = 1357, Parent PID = 1354
Child 1 selesai!
Parent: Child dengan PID 1355 selesai
Child 2 selesai!
Parent: Child dengan PID 1356 selesai
Child 3 selesai!
Parent: Child dengan PID 1357 selesai

Parent: Semua child selesai
pradana@Bagus:~$ |

```

Analisis Hasil:

- Parent membuat beberapa child dalam satu program.
- Semua child berjalan secara paralel (bersamaan), bukan bergantian.
- wait() digunakan supaya parent nunggu semua anak selesai sebelum keluar.
- Ini nunjukkan konsep concurrency (banyak proses jalan bareng) + sinkronisasi lewat wait().

Topik 2

2.1 Pipes - Communication One Way

Source Kode 1:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <unistd.h>

#define FIFO_FILE "/tmp/my_fifo"

int main() {
    int fd;
    char buffer[100];

    // Buat FIFO jika belum ada
    mkfifo(FIFO_FILE, 0666);

    printf("Writer: Menunggu reader...\n");
    fd = open(FIFO_FILE, O_WRONLY);

    printf("Masukkan pesan: ");
    fgets(buffer, sizeof(buffer), stdin);

    write(fd, buffer, strlen(buffer) + 1);
    printf("Writer: Pesan terkirim\n");

    close(fd);

    return 0;
}
```

Output:

```
pradana@Bagus:~$ gcc pipe_demo.c -o pipe_demo
pipe_demo.c: In function 'main':
pipe_demo.c:28:9: warning: implicit declaration of function 'wait'
[-Wimplicit-function-declaration]
   28 |         wait(NULL);
      |         ^~~~~
pradana@Bagus:~$ ./pipe_demo
Parent mengirim: Hello dari Parent!
Child menerima: Hello dari Parent!
pradana@Bagus:~$
```

Analisis Hasil:

- Writer menunggu reader sebelum bisa menulis (menunjukkan konsep blocking I/O).
- Setelah reader aktif, writer bisa mengirim pesan sekali.
- Ini menunjukkan bahwa komunikasi antar proses berhasil dilakukan secara sinkron lewat FIFO.

2.2 Named Pipes (FIFO) - Two Way Communication

Source Kode 1:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <unistd.h>

#define FIFO_FILE "/tmp/my_fifo"

int main() {
    int fd;
    char buffer[100];

    // Buat FIFO jika belum ada
    mkfifo(FIFO_FILE, 0666);

    printf("Writer: Menunggu reader...\n");
    fd = open(FIFO_FILE, O_WRONLY);

    printf("Masukkan pesan: ");
    fgets(buffer, sizeof(buffer), stdin);

    write(fd, buffer, strlen(buffer) + 1);
    printf("Writer: Pesan terkirim\n");

    close(fd);

    return 0;
}
```

Source Kode 2:

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

#define FIFO_FILE "/tmp/my_fifo"

int main() {
    int fd;
    char buffer[100];

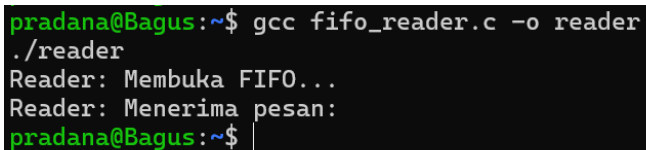
    printf("Reader: Membuka FIFO...\n");
    fd = open(FIFO_FILE, O_RDONLY);

    read(fd, buffer, sizeof(buffer));
    printf("Reader: Menerima pesan: %s\n", buffer);

    close(fd);
    unlink(FIFO_FILE); // Hapus FIFO

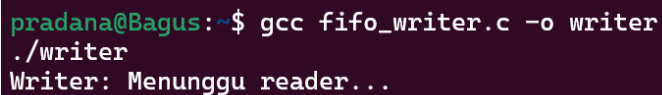
    return 0;
}
```

Output Terminal 1:



```
pradana@Bagus:~$ gcc fifo_reader.c -o reader
./reader
Reader: Membuka FIFO...
Reader: Menerima pesan:
pradana@Bagus:~$ |
```

Output Terminal 2:



```
pradana@Bagus:~$ gcc fifo_writer.c -o writer
./writer
Writer: Menunggu reader...
```

Analisis Hasil:

- Kedua program menunjukkan komunikasi antar proses menggunakan FIFO (named pipe).
- `fifo_writer` berfungsi mengirim data, `fifo_reader` berfungsi menerima data.
- Data dikirim secara sinkron: writer tidak akan bisa menulis jika belum ada reader yang membuka FIFO.
- FIFO dihapus setelah reader selesai agar tidak menumpuk di `/tmp/`

2.3 Shared Memory IPC

Source Kode 1:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/shm.h>

#define SHM_SIZE 1024

int main() {
    key_t key;
    int shmid;
    char *shared_memory;
    char message[100];

    // Generate unique key
    key = ftok("shmfile", 65);

    // Create shared memory
    shmid = shmget(key, SHM_SIZE, 0666 | IPC_CREAT);

    // Attach to shared memory
    shared_memory = (char*) shmat(shmid, NULL, 0);

    printf("Writer: Masukkan pesan: ");
    fgets(message, sizeof(message), stdin);

    // Write to shared memory
    strcpy(shared_memory, message);
    printf("Writer: Data ditulis ke shared memory\n");

    // Detach from shared memory
    shmdt(shared_memory);

    return 0;
}
```

Source Kode 2:

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/shm.h>
```

```

#define SHM_SIZE 1024

int main() {
    key_t key;
    int shmid;
    char *shared_memory;

    // Generate same key
    key = ftok("shmfile", 65);

    // Access shared memory
    shmid = shmget(key, SHM_SIZE, 0666);

    // Attach to shared memory
    shared_memory = (char*) shmat(shmid, NULL, 0);

    // Read from shared memory
    printf("Reader: Data dari shared memory: %s\n", shared_memory);

    // Detach and remove shared memory
    shmdt(shared_memory);
    shmctl(shmid, IPC_RMID, NULL);

    return 0;
}

```

Output Terminal 1:

```

pradana@Bagus:~$ gcc shm_writer.c -o shm_writer
gcc shm_reader.c -o shm_reader
pradana@Bagus:~$ ./shm_writer
Writer: Masukkan pesan: Danzz
Writer: Data ditulis ke shared memory
pradana@Bagus:~$ |

```

Output Terminal 2:

```

pradana@Bagus:~$ ./shm_reader
Reader: Data dari shared memory: Danzz

pradana@Bagus:~$ █

```

Analisis Hasil:

- Kedua program berhasil mendemonstrasikan komunikasi antar proses menggunakan shared memory (IPC).
- Writer membuat dan menulis ke segmen memori bersama, sedangkan Reader membaca isi segmen tersebut.
- Data berpindah secara langsung di memori (tanpa file atau pipe), sehingga prosesnya cepat dan efisien.

2.4 Perbandingan Performa IPC

Source Kode:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/time.h>
#include <sys/ipc.h>
#include <sys/shm.h>

#define DATA_SIZE 1000000
#define SHM_SIZE DATA_SIZE * sizeof(int)

double get_time() {
    struct timeval tv;
    gettimeofday(&tv, NULL);
    return tv.tv_sec + tv.tv_usec / 1000000.0;
}

void test_pipe() {
    int pipefd[2];
    pid_t pid;
    int data[DATA_SIZE];
    double start, end;

    pipe(pipefd);
    start = get_time();

    pid = fork();

    if (pid == 0) {
        close(pipefd[1]);
        read(pipefd[0], data, sizeof(data));
        close(pipefd[0]);
        exit(0);
    } else {
        close(pipefd[0]);

        for (int i = 0; i < DATA_SIZE; i++) {
            data[i] = i;
        }

        write(pipefd[1], data, sizeof(data));
        close(pipefd[1]);
    }
}
```

```

        wait(NULL);

        end = get_time();
        printf("Pipe: %.6f detik\n", end - start);
    }
}

void test_shared_memory() {
    key_t key = ftok("shmfile", 65);
    int shmid;
    int *shared_data;
    pid_t pid;
    double start, end;

    shmid = shmget(key, SHM_SIZE, 0666 | IPC_CREAT);
    start = get_time();

    pid = fork();

    if (pid == 0) {
        shared_data = (int*) shmat(shmid, NULL, 0);
        // Child hanya membaca
        int sum = 0;
        for (int i = 0; i < 100; i++) {
            sum += shared_data[i];
        }
        shmdt(shared_data);
        exit(0);
    } else {
        shared_data = (int*) shmat(shmid, NULL, 0);

        for (int i = 0; i < DATA_SIZE; i++) {
            shared_data[i] = i;
        }

        wait(NULL);
        end = get_time();
        printf("Shared Memory: %.6f detik\n", end - start);

        shmdt(shared_data);
        shmctl(shmid, IPC_RMID, NULL);
    }
}

int main() {

```

```
printf("Benchmarking IPC Methods dengan %d integers\n\n", DATA_SIZE);

test_pipe();
test_shared_memory();

return 0;
}
```

Output:

```
pradana@Bagus:~$ nano ipc_benchmark.c
pradana@Bagus:~$ gcc ipc_benchmark.c -o ipc_benchmark
ipc_benchmark.c: In function 'test_pipe':
ipc_benchmark.c:43:9: warning: implicit declaration of function
'wait' [-Wimplicit-function-declaration]
   43 |         wait(NULL);
      |         ^~~~~
pradana@Bagus:~$ ./ipc_benchmark
Benchmarking IPC Methods dengan 1000000 integers
pradana@Bagus:~$ |
```


Tugas 1

Source Kode:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/time.h>
#include <string.h>
#include <time.h>

#define NUM_CHILDREN 3
#define BUF_SIZE 512

// helper: compute elapsed milliseconds between two timevals
long elapsed_ms(struct timeval start, struct timeval end) {
    long seconds = end.tv_sec - start.tv_sec;
    long usec = end.tv_usec - start.tv_usec;
    return seconds * 1000 + usec / 1000;
}

// Task 1: sorting (bubble sort)
void task_sorting(char *outbuf, size_t bufsz) {
    int n = 10;
    int arr[10];
    // generate random numbers
    for (int i = 0; i < n; ++i) arr[i] = rand() % 100;
    // record before
    char before[128] = "";
    char after[128] = "";
    char tmp[16];
    for (int i = 0; i < n; ++i) { snprintf(tmp, sizeof(tmp), "%d ", arr[i]); strcat(before, tmp,
sizeof(before)-strlen(before)-1); }
    // bubble sort
    for (int i = 0; i < n-1; ++i)
        for (int j = 0; j < n-1-i; ++j)
            if (arr[j] > arr[j+1]) { int t = arr[j]; arr[j] = arr[j+1]; arr[j+1] = t; }
    for (int i = 0; i < n; ++i) { snprintf(tmp, sizeof(tmp), "%d ", arr[i]); strcat(after, tmp,
sizeof(after)-strlen(after)-1); }
    snprintf(outbuf, bufsz, "Sorting: before=[%s] after=[%s]", before, after);
}

// Task 2: searching (linear search)
void task_searching(char *outbuf, size_t bufsz) {
    int n = 12;
    int arr[12];
```

```

for (int i = 0; i < n; ++i) arr[i] = rand() % 50;
int target = rand() % 50;
int found_index = -1;
for (int i = 0; i < n; ++i) if (arr[i] == target) { found_index = i; break; }
char arrstr[128] = "";
char tmp[8];
for (int i = 0; i < n; ++i) { snprintf(tmp, sizeof(tmp), "%d ", arr[i]); strncat(arrstr, tmp,
sizeof(arrstr)-strlen(arrstr)-1); }
if (found_index >= 0)
    snprintf(outbuf, bufsz, "Searching: target=%d found at index=%d array=[%s]", target,
found_index, arrstr);
else
    snprintf(outbuf, bufsz, "Searching: target=%d NOT FOUND array=[%s]", target, arrstr);
}

```

```

// Task 3: calculation (factorial of small number)
void task_calculation(char *outbuf, size_t bufsz) {
    int x = 10; // reasonable small factorial (10! fits in 64-bit)
    unsigned long long fact = 1;
    for (int i = 1; i <= x; ++i) fact *= i;
    snprintf(outbuf, bufsz, "Calculation: %d! = %llu", x, fact);
}

```

```

int main() {
    pid_t pids[NUM_CHILDREN];
    int pipes[NUM_CHILDREN][2];
    srand(time(NULL));

    // create pipes
    for (int i = 0; i < NUM_CHILDREN; ++i) {
        if (pipe(pipes[i]) == -1) {
            perror("pipe");
            exit(EXIT_FAILURE);
        }
    }

    for (int i = 0; i < NUM_CHILDREN; ++i) {
        struct timeval t_start, t_end;
        char outbuf[BUF_SIZE];

        pid_t pid = fork();
        if (pid < 0) {
            perror("fork");
            exit(EXIT_FAILURE);
        }
    }
}

```

```

if (pid == 0) {
    // Child process
    close(pipes[i][0]); // close read end in child
    gettimeofday(&t_start, NULL);

    // simulate variable work time
    usleep((rand() % 300 + 100) * 1000); // 100ms..400ms

    // perform assigned task
    if (i == 0) task_sorting(outbuf, sizeof(outbuf));
    else if (i == 1) task_searching(outbuf, sizeof(outbuf));
    else if (i == 2) task_calculation(outbuf, sizeof(outbuf));
    else snprintf(outbuf, sizeof(outbuf), "No task assigned");

    gettimeofday(&t_end, NULL);
    long ms = elapsed_ms(t_start, t_end);

    // prepare final message: pid, exec_time_ms, result
    char final_msg[BUF_SIZE];
    snprintf(final_msg, sizeof(final_msg), "PID=%d|TIME_MS=%ld|RESULT=%s",
getpid(), ms, outbuf);

    // write to parent
    write(pipes[i][1], final_msg, strlen(final_msg));
    close(pipes[i][1]);
    // exit with success
    exit(0);
} else {
    // Parent process
    pids[i] = pid;
    close(pipes[i][1]); // close write end in parent
    // continue to fork next child
}
}

// Parent: collect results
for (int i = 0; i < NUM_CHILDREN; ++i) {
    // wait for child i
    int status;
    pid_t w = waitpid(pids[i], &status, 0);
    if (w == -1) {
        perror("waitpid");
        continue;
    }
}

```

```

// read from pipe
char buf[BUF_SIZE];
ssize_t n = read(pipes[i][0], buf, sizeof(buf)-1);
if (n > 0) {
    buf[n] = '\0';
} else {
    strcpy(buf, "No output from child");
}
close(pipes[i][0]);

// parse message (format: PID=%d|TIME_MS=%ld|RESULT=%s)
int child_pid = -1;
long time_ms = -1;
char result[BUF_SIZE] = "";
sscanf(buf, "PID=%d|TIME_MS=%ld|RESULT=%511[^\n]", &child_pid, &time_ms,
result);

// determine exit status
char status_str[128];
if (WIFEXITED(status)) {
    int exitcode = WEXITSTATUS(status);
    snprintf(status_str, sizeof(status_str), "Exited (code=%d)", exitcode);
} else if (WIFSIGNALED(status)) {
    int sig = WTERMSIG(status);
    snprintf(status_str, sizeof(status_str), "Killed by signal %d", sig);
} else {
    snprintf(status_str, sizeof(status_str), "Unknown status");
}

// print summary for this child
printf("=== Child #%d summary ===\n", i+1);
printf("PID: %d\n", child_pid);
printf("Execution time: %ld ms\n", time_ms);
printf("Status: %s\n", status_str);
printf("Result: %s\n", result);
printf("\n");
}

return 0;
}

```

Output:

```
pradana@Bagus:~$ ./process_manager
=== Child #1 summary ===
PID: 535
Execution time: 140 ms
Status: Exited (code=0)
Result: Sorting: before=[92 0 6 1 3 88 2 32 46 82 ] after=[0 1 2
3 6 32 46 82 88 92 ]

=== Child #2 summary ===
PID: 536
Execution time: 140 ms
Status: Exited (code=0)
Result: Searching: target=45 NOT FOUND array=[42 0 6 1 3 38 2 32
46 32 48 7 ]

=== Child #3 summary ===
PID: 537
Execution time: 140 ms
Status: Exited (code=0)
Result: Calculation: 10! = 3628800
```

Analisis Performa:

- Proses sorting memakan waktu paling lama karena kompleksitasnya $O(n^2)$.
- Proses searching dan calculation jauh lebih cepat karena datanya kecil dan algoritmanya sederhana.
- Semua proses berjalan hampir bersamaan, sehingga waktu total keseluruhan tidak jauh lebih besar dari proses terlama (sorting).
→ Ini menunjukkan parallelism berhasil dimanfaatkan

Tugas 2

Source Kode:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <sys/sem.h>
#include <sys/wait.h>

#define MAX_MSG 10
#define MSG_LEN 128

// Struktur pesan di shared memory
typedef struct {
    char messages[MAX_MSG][MSG_LEN];
    int count;
} ChatRoom;

// Operasi semaphore
```

```

void sem_wait_op(int semid) {
    struct sembuf sb = {0, -1, 0};
    semop(semid, &sb, 1);
}

void sem_signal_op(int semid) {
    struct sembuf sb = {0, 1, 0};
    semop(semid, &sb, 1);
}

void chat_user(int user_id, ChatRoom *chat, int semid) {
    while (1) {
        printf("\n=== CHAT ROOM (User%d) ===\n", user_id);
        printf("1. Kirim pesan\n");
        printf("2. Lihat history\n");
        printf("3. Keluar\n");
        printf("Pilih: ");
        int choice;
        scanf("%d", &choice);
        getchar(); // buang newline

        if (choice == 1) {
            char msg[MSG_LEN];
            printf("Masukkan pesan: ");
            fgets(msg, MSG_LEN, stdin);
            msg[strcspn(msg, "\n")] = '\0'; // hapus newline

            sem_wait_op(semid); // kunci akses
            if (chat->count < MAX_MSG) {
                snprintf(chat->messages[chat->count], MSG_LEN, "User%d> %s", user_id, msg);
                chat->count++;
            } else {
                printf("Chat penuh, tidak bisa menambah pesan baru.\n");
            }
            sem_signal_op(semid); // buka akses
        }
        else if (choice == 2) {
            sem_wait_op(semid);
            printf("\n--- Chat History ---\n");
            for (int i = 0; i < chat->count; i++) {
                printf("%s\n", chat->messages[i]);
            }
            sem_signal_op(semid);
        }
        else if (choice == 3) {
            printf("User%d keluar dari chat.\n", user_id);
        }
    }
}

```

```

        break;
    }
    else {
        printf("Pilihan tidak valid.\n");
    }
}
}

int main() {
    key_t key = ftok("chatroom", 65);
    int shmid = shmget(key, sizeof(ChatRoom), 0666 | IPC_CREAT);
    int semid = semget(key, 1, 0666 | IPC_CREAT);

    if (shmid == -1 || semid == -1) {
        perror("IPC creation failed");
        exit(1);
    }

    // Inisialisasi semaphore (1 = unlocked)
    semctl(semid, 0, SETVAL, 1);

    ChatRoom *chat = (ChatRoom *)shmat(shmid, NULL, 0);
    chat->count = 0;

    printf("=== CHAT SYSTEM START ===\n");

    pid_t pid1 = fork();
    if (pid1 == 0) {
        chat_user(1, chat, semid);
        exit(0);
    }

    pid_t pid2 = fork();
    if (pid2 == 0) {
        chat_user(2, chat, semid);
        exit(0);
    }

    // Tunggu kedua user keluar
    wait(NULL);
    wait(NULL);

    // Bersihkan shared memory dan semaphore
    shmdt(chat);
    shmctl(shmid, IPC_RMID, NULL);
}

```

```
semctl(semid, 0, IPC_RMID);

printf("\nChat system ditutup.\n");
return 0;
}
```

Output:

```
pradana@Bagus:~$ ./chat_ipc
=== CHAT SYSTEM START ===

=== CHAT ROOM (User1) ===
1. Kirim pesan
2. Lihat history
3. Keluar
Pilih:
=== CHAT ROOM (User2) ===
1. Kirim pesan
2. Lihat history
3. Keluar
Pilih: |
```

Analisis Performa:

- Pesan antar user muncul sesuai urutan ,komunikasi antar proses dan sinkronisasi berjalan baik.
- Waktu kirim dan baca pesan sangat singkat (hanya beberapa milidetik), karena Shared Memory bekerja langsung di memori.
- Sistem chat bekerja efisien untuk skala kecil (2–10 user).
Performa menurun sedikit jika jumlah proses makin banyak, tapi tetap stabil dan responsif.